

Repository Exploration

I inspected the alpha-beta-CROWN repository, focusing on 'complete_verifier/models' and 'complete_verifier/exp_configs'. The models directory contains several benchmark families: MNIST and CIFAR SDP models, ERAN MNIST/CIFAR PyTorch models, OVAL CIFAR models, Marabou CIFAR10 models, toy MNIST networks, CIFAR-10 ResNet models, non-ReLU/custom-operation examples, L2-norm examples, and VNN-COMP benchmark placeholders. The experiment configurations are organized into groups such as 'beta_crown', 'GCP-CROWN', 'BICCOS', 'bab_attack', 'tutorial_examples', and VNN-COMP year folders.

The main difference I observed from Marabou is the workflow. In Marabou, my script loaded a '.nnet' file and directly added input bounds and output inequalities. In alpha-beta-CROWN, the YAML file is the center of the experiment: it names the model loader, dataset loader, norm, epsilon, PGD attack settings, solver parameters, and branch-and-bound timeout. This made the setup more structured, but also stricter because custom keys and paths must match the verifier configuration system.

External Model and Dataset

To make the comparison meaningful, I reused the same model family from Assignment 3. The dataset is EMNIST Digits, an external handwritten-character benchmark loaded through 'torchvision.datasets.EMNIST'. The preprocessing is the same as before: each 28x28 grayscale image is average-pooled to 14x14 and flattened into 196 input values in [0, 1]. The model is 'TinyEmnistMLP', a small ReLU network with architecture '196 -> 32 -> 10'. For Marabou it was exported as '.nnet'; for alpha-beta-CROWN I saved the same architecture as PyTorch '.pt' and also exported ONNX for reference. The selected test sample has true label 0, predicted label 0, and the retrained model reached 95.5% accuracy on the balanced EMNIST test subset.

The verified property is local L-infinity robustness: for all perturbed inputs x' satisfying $\|x' - x\|_\infty \leq \epsilon$ and clipped to [0, 1], the model should keep the original predicted class above every competing class. I ran two epsilons, 0.02 and 0.2, to match the Marabou experiment.

Results and Interpretation

Tool	Epsilon	Result	Status	Runtime
alpha-beta-CROWN	0.02	verified	safe-incomplete	17.44398 s
alpha-beta-CROWN	0.2	falsified	unsafe-pgd	0.43998 s
Marabou	0.02	verified	UNSAT	0.06094 s
Marabou	0.2	falsified	SAT	11.11434 s

For epsilon 0.02, alpha-beta-CROWN first ran PGD and found no violation, then proved the property with initial CROWN bounds. For epsilon 0.2, PGD quickly found an unsafe

adversarial example. This matches the qualitative Marabou behavior: the smaller perturbation was safe, while the larger perturbation admitted a counterexample. The Marabou runtime is reported as the sum of the per-target solve times stored in the Assignment 3 JSON results.

Comparison and Discussion

On this tiny ReLU MLP, both tools were able to answer the same robustness question. Marabou gave a very direct logical encoding style: I could loop over target classes and add inequalities by hand. alpha-beta-CROWN required more configuration work, especially for the custom EMNIST data loader, but once the YAML and loader were correct it produced a compact verified/falsified summary and handled both attack and bound-based verification in one pipeline.

The biggest strength I saw in alpha-beta-CROWN is that it combines fast adversarial search with bound propagation and branch-and-bound. For the unsafe epsilon 0.2 case, it found a counterexample in under half a second. For the safe epsilon 0.02 case, the initial CROWN bounds were already strong enough to prove robustness. The limitation is that custom experiments are sensitive to model/data format details: unsupported config keys, path substitution, and custom loader return shapes caused setup issues that had to be debugged. Overall, alpha-beta-CROWN felt more scalable and experiment-oriented, while Marabou felt more explicit and lower-level.